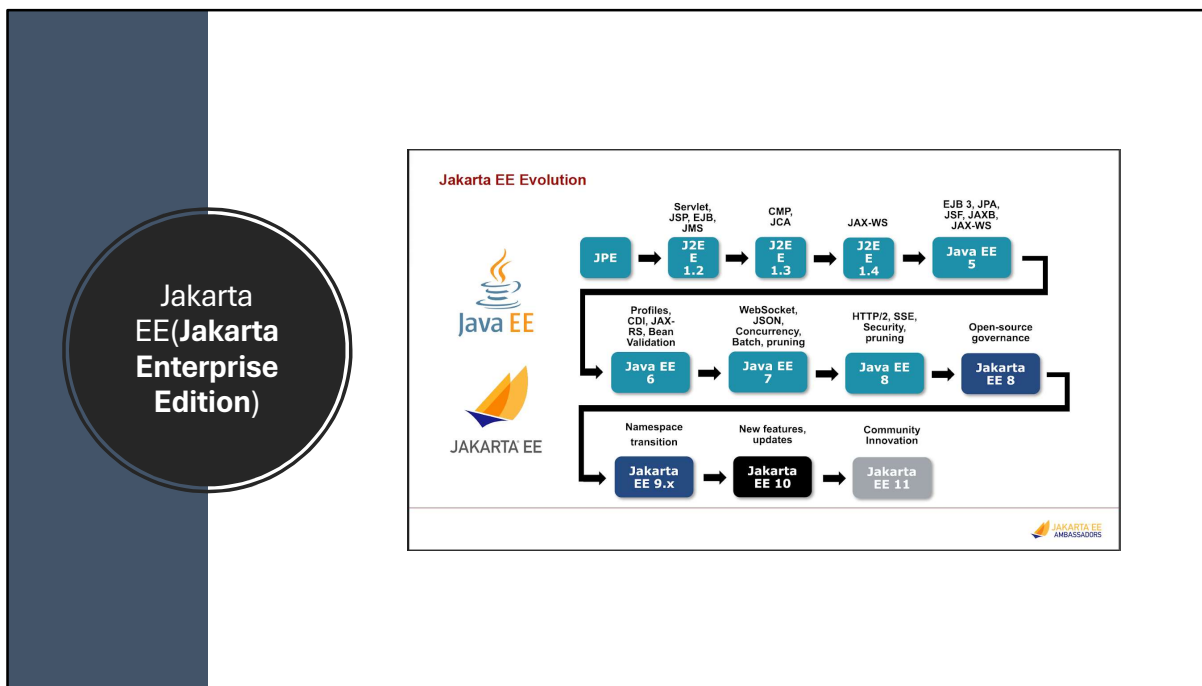


Jakarta Persistence (JPA)

гл.ас. Пенчо Малинов



Революцията в развитието на Java EE и неговата трансформация в Jakarta EE се илюстрира на тази диаграма, която проследява еволюцията от оригиналната Java EE към новата платформа Jakarta EE.

Основни етапи от еволюцията:

1. Java EE (Java Platform, Enterprise Edition)

JPE (Java Platform, Enterprise Edition) е първоначалната платформа за създаване на корпоративни приложения.

Включва компоненти като Servlets, JSP, EJB, JPA, JMS и т.н.

2. Преминане към J2EE и след това към Java EE 5 и версии нагоре

Еволюцията преминава през версии 1.2, 1.3, 1.4, които въвеждат нови функции като Profiles, CDI, Bean Validation, WebSocket, JSON, HTTP/2, сигурност, пречистване (pruning) и други.

Java EE 5 е по-старият стандарт и платформа за създаване на корпоративни приложения в Java.

3. Поява на Java EE 6, 7, 8

В тези версии са добавени нови API и подобрения като динамично управление на контексти, подобрена сигурност, нови компоненти и по-добра поддръжка за уеб услуги.

4. Трансформация към Jakarta EE

Преместването от Java EE към Jakarta EE се случва след прехвърлянето на

управлението от Oracle към Eclipse Foundation.

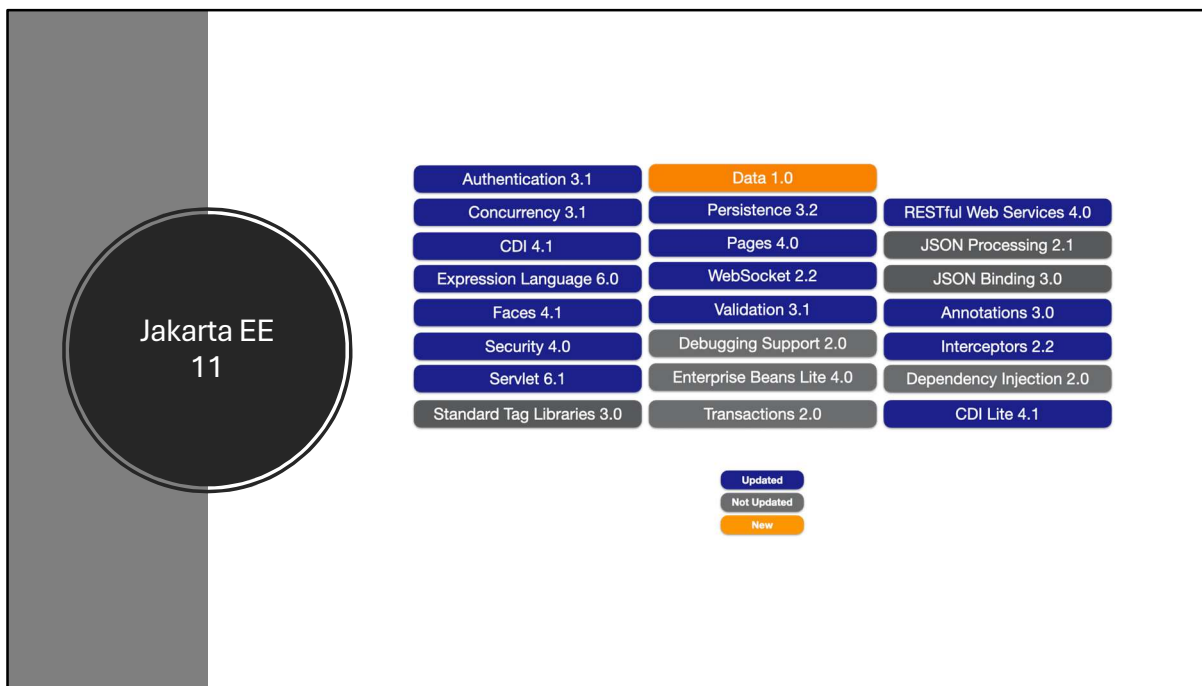
Първоначалната версия **Jakarta EE 8** наследява всички стандарти от Java EE 8.

След това идва **Jakarta EE 10**, която въвежда нови функции и обновления, както и по-нататъшна иновация в общността.

Анонс за следващата версия, Jakarta EE 11, която обещава още повече новости.

Основен акцент

Промяната на името от Java EE към Jakarta EE е свързана с юридически и имиджови причини, тъй като лиценза и търговските права върху марката Java останаха при Oracle, а Eclipse Foundation пое управлението на стандарта под новото име – Jakarta EE.



Основни компоненти (обновени или нови)

Data 1.0

Стандарти за работа с данни в Jakarta EE, включително транзакции и интеграция с бази данни.

Pages 4.0

Технология за създаване на уеб интерфейси с динамично съдържание, често използвана с JSF.

RESTful Web Services 4.0

Стандарти за създаване на REST API, които улесняват обмена на данни между клиент и сървър чрез HTTP.

JSON Processing 2.1

Библиотеки за обработка и манипулация на JSON данни.

JSON Binding 3.0

Технология за автоматично свързване на Java обекти към JSON и обратно.

Annotations 3.0

Поддръжка за използването на annotations, които улесняват конфигурацията и кода.

Interceptor 2.2

Компоненти за управление на междинно поведение на заявките, като логване, сигурност и транзакции.

Старите или по-основните компоненти (обновени или нови)

Authentication 3.1

Управление на автентикацията и сигурността, удостоверяване на потребители.

Concurrency 3.1

Поддръжка за паралелна работа и управление на нишки в приложенията.

CDI 4.1

Contexts and Dependency Injection; основен механизъм за инжектиране на зависимости и управление на житейски цикъл на обекти.

Expression Language 6.0

Език за изрази, който позволява динамично съдържание в страниците и конфигурациите.

Faces 4.1

Компонентна технология за създаване на уеб интерфейси (JSF).

Security 4.0

Поддръжка за сигурност, контрол на достъпа, удостоверяване и авторизация.

Servlet 6.1

Основна технология за създаване на уеб сървлети и обработка на HTTP заявки.

Standard Tag Libraries 3.0

Библиотеки с тагове за улесняване на създаването на уеб съдържание.

Други компоненти (по-старите или по-минимални)

Enterprise Beans Lite 4.0

Облегчена версия на Enterprise Java Beans за бизнес логика.

Transactions 2.0

Управление и координиране на транзакции.

Debugging Support 2.0

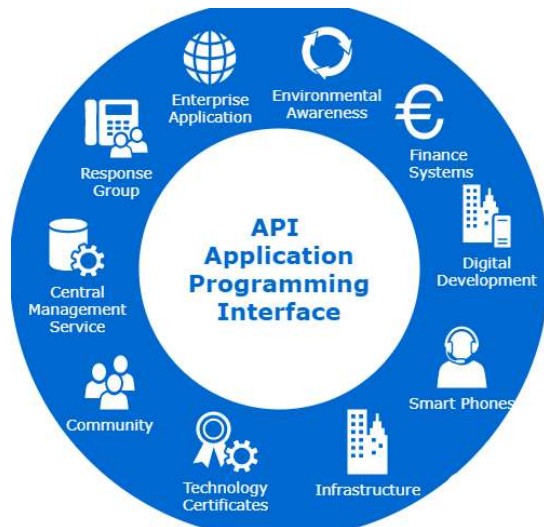
Поддръжка за отстраняване на грешки във приложението.

Dependency Injection 2.0

Механизми за автоматично инжектиране на зависимости.

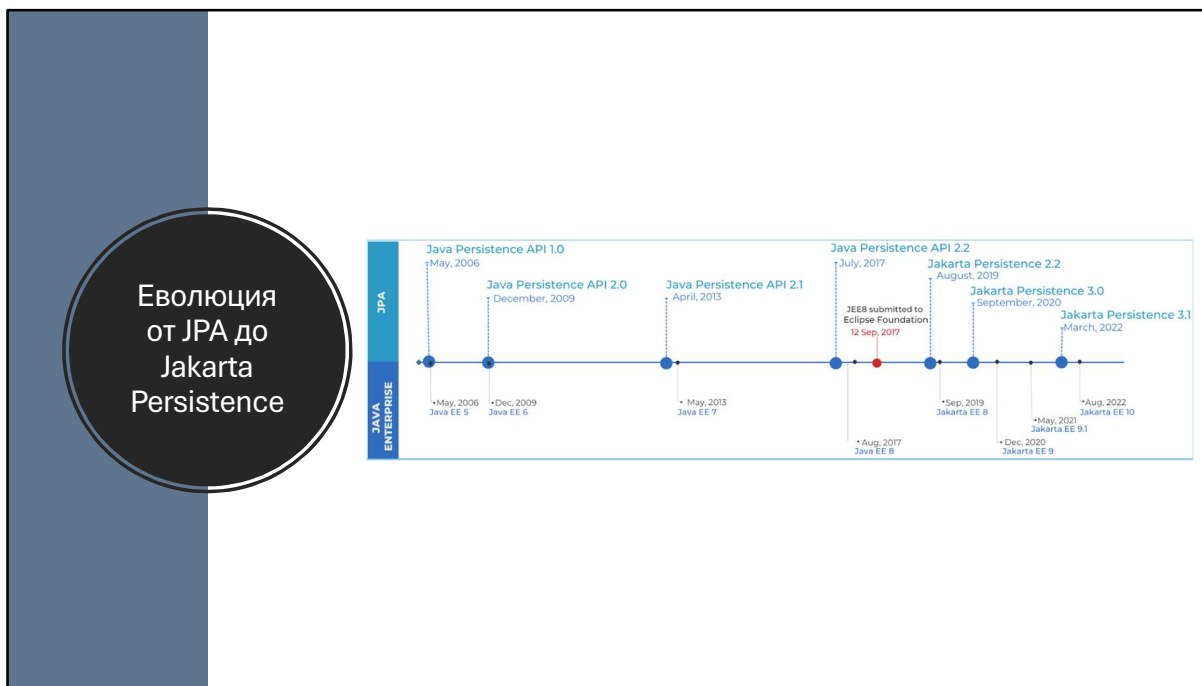
Какво е API

- **API**, или **Application Programming Interface** (интерфейс за програмиране на приложения), е набор от правила, протоколи и инструменти, които позволяват на различни софтуерни системи да комуникират и работят заедно.
- **Обяснение на API:**
- **Интерфейс за взаимодействие:** API определя как една програма може да поиска услуги или данни от друга, като предоставя предварително дефинирани методи, функции и формати за данни.
- **Комуникационен мост:** Например, когато използваш мобилно приложение за проверка на времето, то може да използва API на метеорологична служба, за да вземе актуалната прогноза.



Jakarta Persistence

- **Persistence** в контекста на Jakarta EE е стандартният модул, който се занимава с управлението и обработката на данните в приложенията. Основната технология тук е Jakarta Persistence API (предишното Java Persistence API, JPA), която предоставя обектно-релационно съответствие (ORM — Object-Relational Mapping).
- **Какво е Jakarta Persistence?**
- Jakarta Persistence (JPA) позволява на разработчиците да работят с бази данни чрез Java обекти, вместо директно с SQL заявки. Това улеснява създаването, четенето, актуализирането и изтриването на данни (CRUD операции), като използва анотации, конфигурации и декларативни заявки.
- **Основни функции:**
- **Entity Classes:** Java класове, които представляват таблиците в базата данни.
- **EntityManager:** централният интерфейс за управление на житейския цикъл на entity обектите и за извършване на операции върху базата данни.
- **Query Language:** JPQL (Java Persistence Query Language) за създаване на заявки, които работят с Java обекти.
- **Транзакции:** интегрирана поддръжка за транзакции, гарантираща консистентност на данните.



Еволюцията от JPA към **Jakarta Persistence** е свързана с трансформацията на стандарта за управление на данните в Java от платформата Java EE към новата платформа Jakarta EE. Този преход включва преместване на стандарта и създаване на нова идентичност, като същевременно се запазват основните функции и концепции.

История и развитие:

1. Java Persistence API (JPA)

Първоначално създаден като част от Java EE, JPA стартира като стандартизирана спецификация за ORM (Обектно-реляционно картографиране). Включва основни компоненти като Entity класове, EntityManager, JPQL, транзакционна поддръжка.

Версия 1.0 излиза през 2006 г., следвана от подобрения във версии 2.0, 2.1 и 2.2.

2. Преместване към Jakarta EE

След промяна в управлението на стандартите (от Oracle към Eclipse Foundation), името на платформата се променя от **Java EE** на **Jakarta EE**.

Това означава, че всички компоненти, включително JPA, също трябва да преминат към новата платформа и да бъде преименувани.

3. От JPA в Jakarta Persistence

Преименуването е основното събитие: *Java Persistence API* става **Jakarta Persistence**.

В новата версия, **Jakarta Persistence 3.0** (издадено през 2021 г.), се въвеждат някои промени и подобрения, но принципите остават сходни.

Основната идея е да се запазят стандартите за ORM и взаимодействие с бази данни, докато се преминат към нова идентичност и пространство за имена (javax.persistence става jakarta.persistence).

Ключови моменти:

Преходът е повече името и управлението, отколкото фундаментално изменение в концепциите.

В бъдеще се очакват нови функции и подобрения в рамките на Jakarta Persistence, продължавайки развитието на JPA

Jakarta Persistence – основни предимства

- Стандартен слой за достъп до данни за Java приложения
- Осигурява неутралност спрямо доставчиците
- Безпроблемна интеграция с други спецификации на Jakarta EE
- Основа за корпоративни решения
- Широка подкрепа от общността и доставчиците

1. Като крайъгълен камък на платформата Jakarta EE, Jakarta Persistence предоставя стандартизиран метод за управление на достъпа до бази данни в корпоративни Java приложения. Това гарантира, че приложенията, изградени в екосистемата Jakarta EE, могат да използват последователен, мощен и преносим механизъм за обектно-реляционно картографиране, независимо от основната база данни. Широкото му разпространение означава, че приложенията могат да бъдат премествани или интегрирани с различни Jakarta EE-съвместими сървъри, което значително подобрява гъвкавостта и дългосрочната поддръжка на корпоративните системи.

2. Неутралността спрямо доставчиците е ключово предимство на Jakarta Persistence. Чрез придържане към добре дефинирана спецификация, разработчиците не са обвързани с един доставчик на бази данни или ORM имплементация. Приложенията, кодирани с помощта на Jakarta Persistence, могат да превключват между поддържани имплементации като Hibernate или EclipseLink с минимални усилия. Тази гъвкавост намалява обвързването с доставчици и гарантира, че организациите могат да реагират на съображения за лицензиране, производителност или поддръжка без обширни пренаписвания на код.

3. Jakarta Persistence е тясно интегриран с други спецификации на Jakarta EE, като Jakarta Transactions (JTA) за управление на разпределени транзакции и Jakarta Dependency Injection (CDI) за ефективно управление на компоненти. Тази безпроблемна интеграция предоставя на корпоративните приложения стабилна поддръжка за ACID-съвместими транзакции, инжектиране на зависимости и други критични модели на корпоративен софтуер, което води до по-устойчиви и поддържаеми системи.

4. Като фундаментална технология, Jakarta Persistence поддържа разработването на мащабируеми, високопроизводителни приложения. Функции като кеширане на обекти, лениво зареждане и пакетна обработка подобряват производителността на базата данни и намаляват потреблението на ресурси. В комбинация с функциите от корпоративен клас на Jakarta EE, Jakarta Persistence поддържа изграждането на решения, които обработват огромни количества данни и едновременни потребители, подходящи за организации, изискващи критична надеждност.

5. Голямата общност около Jakarta Persistence означава богатство от ресурси, поддръжка и дългосрочна устойчивост. Редовните актуализации, корекциите за сигурност, най-добрите практики и съвместната култура на отворен код насърчават иновациите и бързото адаптиране към новите изисквания. Предприятията се възползват от колективни знания, стабилни библиотеки и подобрена сигурност – всичко това е жизненоважно за надеждно и дългосрочно разработване на приложения.

Поддържани платформи и среди

- Съвместимост с Java SE и EE
- Работи с основни бази данни
- Поддържа се от Hibernate, EclipseLink
- Готов за облачна работа
- Интегриран с микросървисни архитектури

1. Jakarta Persistence е съвместим както със среди Java Standard Edition (SE), така и с Jakarta EE (преди Java EE). Разработчиците могат да го използват в самостоятелни настолни приложения или в мащабни, сървърно-базирани корпоративни системи. Тази двойна съвместимост осигурява по-голяма гъвкавост, което го прави идеален избор за голямо разнообразие от Java проекти, които изискват надеждни, високопроизводителни слоеве за съхранение на данни във всички типове внедряване.

2. Спецификацията поддържа всички основни релационни бази данни, включително Oracle, PostgreSQL, MySQL, Microsoft SQL Server и други. Благодарение на своя дизайн, имплементациите на Jakarta Persistence могат лесно да бъдат конфигурирани да използват различни бази данни чрез JDBC драйвери и специфични за средата параметри. Това насърчава независимостта и адаптивността на базите данни, когато бизнес нуждите се развиват.

3. Широко използвани имплементации като Hibernate и EclipseLink напълно поддържат Jakarta Persistence, осигурявайки съвместимост с най-новите подобрения на API. Тези доставчици позволяват на разработчиците да избират настройката на производителността и разширените функции, които най-добре отговарят на техните проекти, като същевременно остават в рамките на

стандарта Jakarta. Тази гъвкавост е особено полезна, когато организациите стандартизират специфичен набор от инструменти.

4. Съвременните архитектури на приложения, включително облачно-базирани внедрявания, контейнеризация и микросървиси, се възползват от преносимостта и минималните усилия за конфигуриране, изисквани от Jakarta Persistence. Той е особено подходящ за контейнеризирани Jakarta EE приложения, управлявани от Kubernetes или подобни платформи за оркестрация, където гъвкавостта по време на изпълнение, мащабирането и автоматизираните внедрявания са жизненоважни.

5. Интеграцията с архитектури на микросървиси е осъществена чрез лека, неинвазивна конфигурация и опционална поддръжка на програмен API, като например новия клас `PersistenceConfiguration` в Jakarta Persistence 3.2. Това позволява динамична настройка и избор на база данни по време на изпълнение, отговаряйки на нуждите на микросървиси, които могат да бъдат внедрени и конфигурирани независимо за мащабиране и устойчивост.



Основни ORM концепции

- Entities и mappings
- Persistence context
- Transactions management
- JPQL и Criteria API
- Кеширане и оптимизация на производителността

1. Една от централните идеи на Jakarta Persistence е концепцията за обекти и техните съпоставяния с таблици от база данни. Обектите са обикновени Java обекти (POJO), маркирани с анотации като @Entity, и всеки екземпляр съответства на ред в таблица от база данни. Тези съпоставяния позволяват на разработчиците да обработват Java обекти директно, делегирайки детайлите за трансформацията и персистирането на имплементацията на Jakarta Persistence и по този начин позволявайки данните и бизнес логиката да останат тясно свързани в кода.

2. Контекстът на персистиране управлява състоянието на всички екземпляри на обекти в рамките на един транзакционен обхват. Той следи операциите по жизнения цикъл на обекти, като създаване, актуализиране и премахване, и осигурява съгласуваност на данните между Java приложението и основната база данни. Чрез управление на състоянията на обектите и синхронизацията, Jakarta Persistence помага за налагането на транзакционни граници и подобрява програмния контрол върху промените в данните.

3. Управлението на транзакциите е тясно интегрирано, което позволява на разработчиците да обгръщат операциите с базата данни в атомарни единици работа. Jakarta Persistence функционира безпроблемно с Jakarta Transactions

(JTA), което прави възможно координирането на промените в множество ресурси и гарантирането на целостта на данните дори в разпределени или мултибазни среди. API за управление на транзакции предоставят на разработчиците прецизен контрол, като същевременно насърчават най-добрите практики за надеждни и мащабируеми приложения.

4. Jakarta Persistence Query Language (JPQL) и Criteria API предоставят безопасни за типа, обектно-ориентирани механизми за изграждане на заявки. JPQL позволява писане на заявки към база данни, използвайки имена и полета на Java обекти, което прави заявките по-четими и лесни за поддръжка. Criteria API улеснява динамичното, програмно създаване на заявки, намалявайки риска от синтактични грешки и поддържайки разширени функции като филтриране, сортиране и обединяване на обекти.

5. Кеширането и оптимизацията на производителността са вградени в Jakarta Persistence. Поддръжката на кеш от L2 (второ ниво), lazy loading и кеширането на резултатите от заявките заедно намаляват натоварването на базата данни, повишават производителността на приложенията и намаляват времето за реакция. Имплементациите могат допълнително да оптимизират бавните заявки, да управляват стратегиите за извличане и пакетните операции, за да отговорят на изискванията за мащабиране на големи корпоративни и уеб приложения.

Какво е Entity?

- Java клас, съпоставен с таблица/и в базата данни
- Анотиран с @Entity
- Трябва да има първичен ключ (@Id)
- Поддържа наследяване

```
--
33 // Наследник - Dog
34 @Entity
35 @DiscriminatorValue("Dog")
36 public class Dog extends Animal {
37
38     private String breed;
39 }
--

11 @Entity
12 public class Person {
13     @Id
14     @GeneratedValue(strategy = GenerationType.IDENTITY)
15     private Long id;
16     private String name;
17 }
--

3 // Базов клас - може да е абстрактен
4 @Entity
5 @Inheritance(strategy = InheritanceType.SINGLE_TABLE)
6 @DiscriminatorColumn(name = "animal_type")
7 public abstract class Animal {
8
9     @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     private Long id;
12 }

49 // Наследник - Cat
50 @Entity
51 @DiscriminatorValue("Cat")
52 public class Cat extends Animal {
53
54     private String favoriteToy;
55 }
```

1. В Jakarta Persistence, обектът е лек Java клас – известен като Plain Old Java Object (POJO) – който е съпоставен с таблица в релационна база данни чрез използването на анотации или XML конфигурация. Всеки клас, анотиран с @Entity, представлява отделна таблица, а неговите свойства или полета представляват съответните колони на таблицата, което позволява интуитивен обектно-базиран достъп до релационни данни.

2. Обектите трябва да включват уникален идентификатор или първичен ключ, дефиниран от анотацията @Id на едно от неговите полета или свойства. Този ключ е жизненоважен за уникалното идентифициране на записи по време на операции с базата данни и поддържане на състоянията на жизнения цикъл на обектите в контекста на персистиране. Използването на първични ключове позволява ефективно търсене, актуализиране и изтриване на обекти чрез Java код.

3. Обектите поддържат наследяване, което позволява полиморфни модели на данни. Със стратегии за наследяване като SINGLE_TABLE, JOINED или TABLE_PER_CLASS, Jakarta Persistence може да управлява абстрактни базови класове или споделени полета, поддържайки обектно-ориентирани принципи, като същевременно съхранява данните ефективно. Това е от решаващо значение

за сложни модели на домейни и помага за поддържане на съответствие между схемите на базите данни и йерархиите на Java класовете.

4. Чрез активиране на CRUD (създаване, четене, актуализиране, изтриване) операции за обекти, Jakarta Persistence намалява сложността на взаимодействието с базата данни. Разработчиците могат да създават, променят и премахват обекти, просто като извикват методи на EntityManager, като например persist, merge и remove. Рамката управлява генерирането на SQL, подготовката на оператори и разграничаването на транзакциите автоматично.

5. Обектите също така са разширяеми чрез анотации, които дефинират правила за валидиране, опции за картографиране и обратни извиквания за жизнения цикъл. Разработчиците могат да добавят функции като автоматична валидация преди запазване на данни, дефиниране на ограничения на базата данни и слушатели на обекти за прихващане на събития от жизнения цикъл (напр. преди запазване, след актуализиране). Това подобрява модулността, възможността за повторно използване и поддръжката на бизнес логиката.

Основни анотации

@Entity

@Table

@Id

@Column

@JoinColumn

Анотации за жизнения цикъл - @PrePersist, @PostPersist, @PreUpdate, @PreRemove и @PostLoad

1. Анотацията @Entity маркира Java клас като персистентен обект, което прави класа подходящ за ORM съпоставяне. Тя сигнализира на рамката Jakarta Persistence, че класът трябва да се управлява в контекста на персистентност на данните. Обикновено съответната таблица в базата данни е кръстена на този клас, освен ако не е презаписана с други анотации, установявайки жизненоважна връзка между обектния модел и релационната схема.
2. Анотацията @Table ви позволява да персонализирате как даден обект се съпоставя с таблица в базата данни, позволявайки задаване на имена на таблици, схема, уникални ограничения или индекси. Това е особено полезно при интегриране на Java код със съществуващи схеми на бази данни, тъй като осигурява гранулиран контрол върху структурата на таблиците, индексването и спазването на стандартите за именуване в различни организационни бази данни.
3. Анотацията @Id определя поле или свойство като първичен ключ на обекта, който уникално идентифицира всеки запис в таблицата в базата данни. Разработчиците могат да избират различни типове данни за първичния ключ и често комбинират @Id с @GeneratedValue, за да позволят автоматично генериране на ключове от основната база данни. Това е от основно значение за поддържане на целостта на данните и ефективни CRUD операции върху обекти.
4. Анотациите @Column и @JoinColumn помагат за уточняване на детайлно

съпоставяне на ниво колона и връзки между обекти. @Column позволява персонализиране на имената, типовете, дължините и ограниченията на колоните, докато @JoinColumn установява връзки с външен ключ, поддържайки асоциации като едно към едно или много към едно. Тези съпоставяния позволяват прецизно съгласуване с изискванията на базата данни и оптимален дизайн на заявките.

5. Анотации за жизнения цикъл, като @PrePersist, @PostPersist, @PreUpdate, @PreRemove и @PostLoad, предоставят връзки към събития, свързани с операции с обекти. Разработчиците могат да ги използват, за да задействат валидиране, регистриране или персонализирана логика в стратегически точки от процеса на запазване на данни, като гарантират, че междусекторните проблеми (като например одит) се обработват последователно и автоматично в рамките на операциите с обекти на приложението.

Анотацията @Entity е жизненоважна в Jakarta Persistence, трансформирайки обикновен Java клас в персистентен обект, управляван от ORM рамката. След като бъде анотиран, класът става част от persistence единица и се съпоставя с таблица в базата данни, което позволява на разработчиците да обработват сложни структури от данни в рамките на познатия език Java. Конвенцията е името на таблицата да съвпада с името на класа, освен ако не е посочено друго, което прави управлението на обекти лесно. Всеки клас обекти трябва да има конструктор по подразбиране (без аргументи). Това е така, защото Jakarta Persistence трябва да създава динамично инстанции на обекти по време на операции по заявка и извличане. Създаването на този конструктор гарантира правилното функциониране на ORM слоя и поддържа прозрачно зареждане на данни, особено когато обектите се извличат на партии. Ако е необходимо, разработчиците могат да персонализират името на таблицата в базата данни, използвайки анотацията @Table. Това е особено важно при интегриране със стари бази данни или припокриване със съществуващи схеми. Независимо дали таблицата е кръстена на класа на обекта или е посочена изрично, съпоставянето трябва да е ясно и да съответства на конвенциите на базата данни, за да се намали объркването или грешките. Обектите работят ръка за ръка с други ORM анотации, като например @Id за уникална идентификация, @Column за картографиране на ниво поле и асоциативни анотации за взаимоотношения. Те колективно определят структурата, целостта и навигацията на постоянните обекти, осигурявайки стабилни принципи на проектиране, ориентирани към домейна. Чрез използването на @Entity и свързани анотации, разработчиците могат ефективно да картографират обекти на домейна към релационни структури, да минимизират шаблонния код и да създават по-чиста и по-поддържана логика на бизнес приложенията. Обектите опростяват персистентността, насърчават обектно-ориентирани модели и правят Jakarta Persistence привлекателна опция за сложни, модерни Java приложения.

@Id

В JPA (Jakarta Persistence API) анотацията @Id е една от най-важните и основни анотации, използвана за отбелязване на полето в ентити класа, което служи като **първичен ключ** (primary key) в базата данни.



@GeneratedValue — използва се за автоматично генериране на стойността на полето @Id. Възможните стратегии за генериране са:

AUTO – изборът се оставя на провайдъра на JPA (най-често Hibernate)

IDENTITY – базата данни автоматично генерира стойността (например auto-increment)

SEQUENCE – използва се базова последователност (sequence) в базата данни

TABLE – използва отделна таблица за проследяване на идентификаторите

```
@Entity
public class User {
    @Id
    @GeneratedValue(strategy =
    GenerationType.IDENTITY)
    private Long id;
    private String name;
    // гетъри и сетъри
}
```

@Column

- @Column е анотация, която се използва за дефиниране и конфигуриране на мапинга между дадено поле (или свойство) в Java ентити клас и конкретна колона в базата данни. Контролира името на колоната, типа ѝ, дали може да приема null стойности, дължината на колоната, уникалността и други ограничения.

```
1 @Entity
2 public class User {
3
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     @Column(name = "username", nullable = false, length = 50, unique = true)
9     private String userName;
10
11     @Column(name = "email_address", nullable = false)
12     private String email;
```

name	Определя името на колоната в базата данни (по подразбиране името на полето)
nullable	Определя дали колоната може да има стойност <code>null</code> (по подразбиране - <code>true</code>)
length	Максималната дължина на колоната (важи за текстови типове като <code>VARCHAR</code>)
unique	Определя уникалността на стойността в колоната (стойността трябва да е уникална)
insertable	Дали колоната да бъде част от SQL INSERT операции (по подразбиране <code>true</code>)
updatable	Дали колоната може да се променя по време на UPDATE операции (по подразбиране <code>true</code>)
columnDefinition	Позволява директно да зададете SQL дефиниция за колоната
precision и scale	Контролира точността и мащаба при числа от тип <code>BigDecimal</code> и др.

@OneToOne

Анотацията **@OneToOne** в JPA (Jakarta Persistence API) се използва за дефиниране на еднопосочна или двупосочна връзка „един към един“ между две ентити (същности). Това означава, че всеки обект от първия тип е свързан с точно един обект от втория тип, и обратно



Какво представлява @OneToOne?

Свързва две ентити, така че между тях да съществува взаимоотношение „един към един“.

Пример: Връзката между **Потребител** и **Паспорт** – всеки потребител има точно един паспорт и всеки паспорт принадлежи на точно един потребител.



Основни характеристики:

Може да е **еднопосочна** или **двупосочна** връзка.

В базата данни обикновено се използва **външен ключ** или **споделен първичен ключ**.

Връзката може да бъде зареждана **ляниво (lazy)** или **жадно (eager)**.

@OneToOne

Пример за двупосочна връзка:

```
1 @Entity
2 public class User {
3
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     @OneToOne(mappedBy = "user")
9     private Passport passport;
10
11     // гетъри и сетъри
12 }
13
14 @Entity
15 public class Passport {
16
17     @Id
18     @GeneratedValue(strategy = GenerationType.IDENTITY)
19     private Long id;
20
21     private String number;
22
23     @OneToOne
24     @JoinColumn(name = "user_id")
25     private User user;
26
27     // гетъри и сетъри
28 }
```

Пример за еднопосочна връзка:

```
1 @Entity
2 public class User {
3
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     @OneToOne
9     @JoinColumn(name = "passport_id") // външен ключ в таблицата User
10    private Passport passport;
11
12    // гетъри и сетъри
13 }
14
15 @Entity
16 public class Passport {
17
18     @Id
19     @GeneratedValue(strategy = GenerationType.IDENTITY)
20     private Long id;
21
22     private String number;
23
24     // гетъри и сетъри
25 }
```

@OneToOne

- Често използвани параметри в @OneToOne:

ПАРАМЕТАР	ЗНАЧЕНИЕ
<code>mappedBy</code>	Определя собственика на връзката (обратно поле)
<code>cascade</code>	Дефинира кои операции се насочват към свързаното ентити (например <code>CascadeType.ALL</code>)
<code>fetch</code>	Контролира зареждането — <code>FetchType.LAZY</code> или <code>FetchType.EAGER</code>
<code>optional</code>	Дали връзката е задължителна (<code>false</code>) или опционална (<code>true</code>)

Кога да използваме @OneToOne?

- Когато всяка инстанция от едната страна е свързана само с една от другата (напр. профил към потребител, паспорт към човек).
- Когато трябва да разделим данни в различни таблици, но логическата връзка е 1:1.

@OneToMany

Анотацията **@OneToMany** в JPA (Jakarta Persistence API) служи за дефиниране на връзка тип „един към много“ между две ентити (същности). Това означава, че една инстанция на първия клас (родител) може да бъде свързана с много инстанции на втория клас (деца).

Какво представлява @OneToMany?

- Свързва един обект с колекция от други обекти.
- Пример: Един **Автор** може да има много **Книги** — авторът е „един“, а книгите са „много“.
- В базата данни връзката обикновено се реализира чрез външен ключ в таблицата на детето.

Основни характеристики:

- Обикновено в JPA се дефинира на страната на родителя и сочи към списък или друг колекция тип (напр. List, Set).
- Връзката може да бъде **еднопосочна** или **двупосочна** с помощта на mappedBy.

@OneToMany

Пример за еднопосочна връзка

```
1 @Entity
2 public class Author {
3
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     private String name;
9
10    @OneToMany
11    @JoinColumn(name = "author_id") // външен ключ в таблицата Book
12    private List<Book> books = new ArrayList<>();
13
14    // гетъри и сетъри
15 }
16
17 @Entity
18 public class Book {
19
20     @Id
21     @GeneratedValue(strategy = GenerationType.IDENTITY)
22     private Long id;
23
24     private String title;
25
26     // гетъри и сетъри
27 }
```

В този случай таблицата **Book** съдържа колоната `author_id`, която сочи към автора.

Пример за двупосочна връзка @OneToMany — @ManyToOne:

```
1 @Entity
2 public class Author {
3
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     private String name;
9
10    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval = true)
11    private List<Book> books = new ArrayList<>();
12
13    // гетъри и сетъри
14 }
15
16 @Entity
17 public class Book {
18
19     @Id
20     @GeneratedValue(strategy = GenerationType.IDENTITY)
21     private Long id;
22
23     private String title;
24
25     @ManyToOne
26     @JoinColumn(name = "author_id")
27     private Author author;
28
29     // гетъри и сетъри
30 }
```

Тук `mappedBy = "author"` показва, че връзката се управлява от полето `author` в класа `Book`.

@OneToMany

- Често използвани параметри на @OneToMany:

ПАРАМЕТЪР	ЗНАЧЕНИЕ
<code>mappedBy</code>	Указва собственика на връзката (страна, където е FK)
<code>cascade</code>	Определя кои операции се пренасят върху свързаните ентити (напр. <code>CascadeType.ALL</code>)
<code>fetch</code>	Контролира зареждането - по подразбиране е <code>LAZY</code>
<code>orphanRemoval</code>	Автоматично изтрива „осиротели“ деца, ако са премахнати от колекцията

Защо и кога да използваме @OneToMany?

- Когато има естествено отношение „един към много“, напр. един клиент има много поръчки.
- Позволява лесно управление на колекция от свързани обекти чрез JPA.
- Помага при изграждането на модела на данни и улеснява заявки и навигация между свързаните ентити.



@ManyToMany

- Анотацията **@ManyToMany** в JPA (Jakarta Persistence API) се използва за дефиниране на връзка „много към много“ между две ентити (същности). Това означава, че едно инстанциране на първия клас може да бъде свързано с много инстанции на втория, и обратно.
- **Какво представлява @ManyToMany?**
 - Свързва две ентити в двупосочна или едноразична Many-to-Many релация.
 - Пример: Много студенти могат да посещават много курса, и всеки курс може да има много студенти.
 - В базата данни тази връзка се реализира чрез междинна (join) таблица, която съдържа външни ключове към двете таблици.
- **Основни характеристики:**
 - Връзката обикновено се дефинира с колекция (Set, List) от свързаните обекти в двете ентити.
 - Само едната страна на връзката е „собственик“ на връзката и управлява join таблицата, а другата използва mappedBy.

@ManyToMany

Пример за двупосочна ManyToMany връзка:

```
1 @Entity
2 public class Student {
3
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     private String name;
9
10    @ManyToMany
11    @JoinTable(
12        name = "student_course",
13        joinColumns = @JoinColumn(name = "student_id"),
14        inverseJoinColumns = @JoinColumn(name = "course_id")
15    )
16    private Set<Course> courses = new HashSet<>();
17
18    // гетъри и сетъри
19 }
20 |
21 @Entity
22 public class Course {
23
24     @Id
25     @GeneratedValue(strategy = GenerationType.IDENTITY)
26     private Long id;
27
28     private String title;
29
30     @ManyToMany(mappedBy = "courses")
31     private Set<Student> students = new HashSet<>();
32
33     // гетъри и сетъри
34 }
```

@JoinTable дефинира междинната таблица student_course.

joinColumns и inverseJoinColumns определят външните ключове към съответните таблици.

В класа Course се използва mappedBy, за да се посочи, че собственик на връзката е Student

@ManyToMany

- Параметри на @ManyToMany:

ПАРАМЕТЪР	ОПИСАНИЕ
<code>mappedBy</code>	Определя името на полето, което владее връзката
<code>cascade</code>	Определя кои операции да се каскадират върху свързаните обекти
<code>fetch</code>	Определя начина на зареждане - <code>LAZY</code> (по подразбиране) или <code>EAGER</code>

Кога да използваме @ManyToMany?

- Когато между две ентити съществува взаимна т.нар. „много към много“ връзка.
- Например, студенти и курсове, автори и книги (ако една книга има много автори, а един автор има много книги) и други.

Важни неща за имате предвид:

- Винаги има таблица „връзка“/join table, която съхранява асоциациите.
- За по-голям контрол върху join таблицата може да се използва отделно ентити, която да представя връзката (т.нар. „асоцииращо ентити“).

Cascade types

- В JPA (Jakarta Persistence API) **каскадите** (cascade types) определят как определени операции, изпълнени върху една ентити, се пренасят автоматично върху свързаните с нея ентитита. Това улеснява управлението на свързани данни и поддържа консистентността им.

- Основни типове каскади (CascadeType):**

ТИП КАСКАДА	ОПИСАНИЕ
PERSIST	При запазване на главната ентити, свързаните ентитита също се запазват (persist).
MERGE	При сливане (merge) на главната ентити, свързаните ентитита също се сливат (merge).
REMOVE	При изтриване на главната ентити, свързаните ентитита също се изтриват (remove).
REFRESH	При опресняване (refresh) на главната ентити, свързаните ентитита също се опресняват от базата данни.
DETACH	При отделяне (detach) на главната ентити от контекста на перзистънс, свързаните ентитита също се отделят.
ALL	Включва всички горепосочени операции (PERSIST, MERGE, REMOVE, REFRESH, DETACH).

Cascade types

Кога и как се използват каскадите?

- Когато имате свързани ентитета (например Parent и Child), каскадите позволяват автоматичното прилагане на операции върху децата, когато се извършват операции върху родителя.
- Например, ако зададете `cascade = CascadeType.PERSIST` върху свързаните деца, когато запишете родителя, децата също ще се запишат автоматично.
- Без каскади трябва ръчно да записвате или изтривате всяка свързана ентити, което усложнява кода.

Cascade types

Какво да внимаваме:

- REMOVE трябва да се използва внимателно, защото може да изтрие много данни, ако не е правилно конфигурирана.
- Понякога е по-добре да не използвате каскадните операции за изтриване, а да управлявате изтриването ръчно, особено при сложни връзки.
- Параметърът `orphanRemoval=true` е полезен, когато искате да изтривате „осиротели“ свързани обекти, които са премахнати от колекцията в родителя.

Тук, ако направите `persist`, `merge` или `remove` на `Parent`, тези операции ще се прилагат и върху всички обекти в списъка `children`.

```
1 @Entity
2 public class Parent {
3
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     @OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
9     private List<Child> children = new ArrayList<>();
10
11     // гетъри и сетъри
12 }
```

orphanRemoval = true при връзка
@OneToMany в JPA:

Обяснение:

• Когато даден Child обект бъде премахнат от списъка children в Parent чрез removeChild() (т.е. премахнат от колекцията и връзката към родителя се зададе на null), JPA автоматично ще изтрие този Child от базата данни благодарение на orphanRemoval = true.

• Ако не използвате orphanRemoval=true, премахването на Child от колекцията няма да доведе до изтриването му от базата, а той ще остане „осиротял“ (без връзка към Parent).

```
1 Parent parent = entityManager.find(Parent.class, 1L);
2 Child childToRemove = parent.getChildren().get(0);
3
4 parent.removeChild(childToRemove); // премахваме детето от списъка и нулираме връзката
5
6 entityManager.getTransaction().begin();
7 entityManager.persist(parent);
8 entityManager.getTransaction().commit(); // Child автоматично ще бъде изтрит от базата данни
```

```
1 @Entity
2 public class Parent {
3
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     private String name;
9
10    @OneToMany(mappedBy = "parent", cascade = CascadeType.ALL, orphanRemoval = true)
11    private List<Child> children = new ArrayList<>();
12
13    // return и set/get
14
15    public void addChild(Child child) {
16        children.add(child);
17        child.setParent(this);
18    }
19
20    public void removeChild(Child child) {
21        children.remove(child);
22        child.setParent(null);
23    }
24 }
25
26 @Entity
27 public class Child {
28
29     @Id
30     @GeneratedValue(strategy = GenerationType.IDENTITY)
31     private Long id;
32
33     private String name;
34
35     @ManyToOne
36     @JoinColumn(name = "parent_id")
37     private Parent parent;
```

Наследяване на ентити (Entity Inheritance)

В JPA (Jakarta Persistence API) наследяването на ентити позволява да моделирате обекти в Java с класическата обектно-ориентирана концепция за наследяване, и същевременно те да се съхраняват в релационна база данни. Това е много полезно при сложни модели на данни, където различни видове обекти споделят общи атрибути и поведение.

Начини за реализиране на наследяване в JPA

JPA поддържа три основни стратегии за съхранение на наследявани ентити в базата данни:

1. SINGLE_TABLE (Една таблица за всички класове)

Всички класове от йерархията се съхраняват в една и съща таблица.

- Използва се `@Inheritance(strategy = InheritanceType.SINGLE_TABLE)` върху базовия клас.
- В таблицата има специална колона (`@DiscriminatorColumn`), която указва типа на всеки ред.

```
1 @Entity
2 @Inheritance(strategy = InheritanceType.SINGLE_TABLE)
3 @DiscriminatorColumn(name = "animal_type")
4 public abstract class Animal { ... }
5
6 @Entity
7 @DiscriminatorValue("Dog")
8 public class Dog extends Animal { ... }
9
10 @Entity
11 @DiscriminatorValue("Cat")
12 public class Cat extends Animal { ... }
```

Наследяване на ентити (Entity Inheritance)

2.JOINED (Таблица за всеки клас)

Всеки клас от йерархията се съхранява в своя таблица, а таблиците са свързани чрез join по първичен ключ.

•Използва се `@Inheritance(strategy = InheritanceType.JOINED)` върху базовия клас.

```
1 @Entity
2 @Inheritance(strategy = InheritanceType.JOINED)
3 public abstract class Animal { ... }
4
5 @Entity
6 public class Dog extends Animal { ... }
7
8 @Entity
9 public class Cat extends Animal { ... }
```

Наследяване на ентити (Entity Inheritance)

3.TABLE_PER_CLASS (Отделна таблица за всеки клас)

Всяка реализация в йерархията има своя собствена таблица с всички полета (включително от базовия клас).

- Използва се `@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)`
- Няма таблица за базовия клас

```
1 @Entity
2 @Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
3 public abstract class Animal { ... }
4
5 @Entity
6 public class Dog extends Animal { ... }
7
8 @Entity
9 public class Cat extends Animal { ... }
```

Наследяване на ентити (Entity Inheritance)

Абстрактни класове и @MappedSuperclass

•**@MappedSuperclass**: Ако имате общи полета и логика, но не искате класът да бъде таблица или да се извършва запазване директно за него, може да използвате @MappedSuperclass. В този случай полетата наследяват класовете-потомци, но няма отделна таблица за базовия клас.

```
1 @MappedSuperclass
2 public abstract class BaseEntity {
3     @Id
4     @GeneratedValue
5     private Long id;
6
7     private LocalDateTime createdAt;
8
9     // гетъри и сетъри
10 }
```

Тогава другите ентити наследяват от BaseEntity, но няма таблица, която да съхранява самия BaseEntity.

Наследяване на ентити (Entity Inheritance)

Кога да използваме наследяване при ентити

- Когато множество класове имат общи атрибути и поведение, които искате да моделирате по обектно-ориентиран начин.
- За по-удобно управление и преизползване на кода.
- Когато искате да съхранявате различни типове обекти, но да ги третирате в единна структура, например с полиморфни заявки.

Обобщение

- JPA позволява лесно моделиране на класическо наследяване.
- Има три основни стратегии за съхранение в БД - SINGLE_TABLE, JOINED, TABLE_PER_CLASS.
- @MappedSuperclass е удобен за споделяне на общи атрибути без създаване на таблица.
- Изборът на стратегия зависи от нуждите за производителност, нормализация и видимост на данните.

Конфигурации

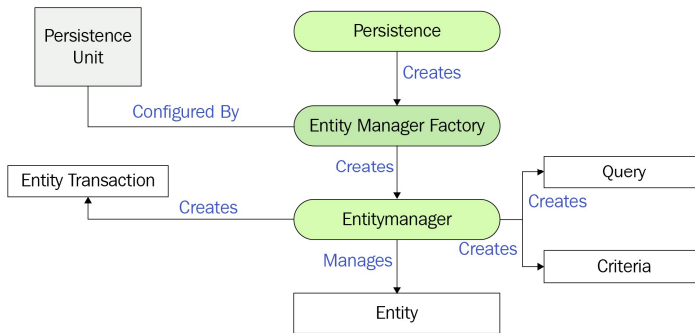
Конфигурациите за **Persistence** в JPA (Jakarta Persistence API) определят начина, по който приложението взаимодейства с базата данни и настройките за съхраняване, извличане и управление на данните. Обикновено това се реализира чрез конфигурационен файл, а именно **persistence.xml**.

Основен конфигурационен файл: **persistence.xml**

Този XML файл се поставя в папката META-INF на проекта и съдържа основните настройки за създаване на EntityManagerFactory и управлението на персистънс контекста.

```
1 <persistence xmlns="http://java.sun.com/xml/ns/persistence"
2   version="2.2">
3   <persistence-unit name="myPersistenceUnit" transaction-type="RESOURCE_LOCAL">
4     <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
5     <class>com.example.MyEntity</class>
6   <properties>
7     <property name="javax.persistence.jdbc.url" value="jdbc:mysql://localhost:3306/mydb"/>
8     <property name="javax.persistence.jdbc.user" value="dbuser"/>
9     <property name="javax.persistence.jdbc.password" value="dbpassword"/>
10    <property name="javax.persistence.jdbc.driver" value="com.mysql.cj.jdbc.Driver"/>
11
12    <!-- Hibernate специфични свойства -->
13    <property name="hibernate.dialect" value="org.hibernate.dialect.MySQL8Dialect"/>
14    <property name="hibernate.hbm2ddl.auto" value="update"/>
15    <property name="hibernate.show_sql" value="true"/>
16  </properties>
17 </persistence-unit>
18 </persistence>
```

JPA архитектура



Основни компоненти в persistence.xml

- **<persistence-unit>**: Основен елемент, който дефинира една конфигурация за връзка с база данни (може да има повече от един).
- Атрибути:
 - name: име на юнита.
 - transaction-type: RESOURCE_LOCAL (локални транзакции) или JTA (JTA транзакции).
- **<provider>**: Провайдер на JPA имплементацията (например org.hibernate.jpa.HibernatePersistenceProvider).
- **<class>**: Ентити класове, които участват в този юнит. Не е задължително
- **<properties>**: Конфигурационни свойства, за връзка с базата данни и настройките на ORM слоя:
 - javax.persistence.jdbc.url: URL за връзка с базата.
 - javax.persistence.jdbc.user: потребител.
 - javax.persistence.jdbc.password: парола.
 - javax.persistence.jdbc.driver: драйвер на БД.
 - hibernate.dialect: диалект на SQL (зависимост от базата).
 - hibernate.hbm2ddl.auto: автоматично управление на схемата (update, create, validate).
 - hibernate.show_sql: показва SQL заявките в конзолата.

JPA конфигурации в Spring Boot

- **Автоматична конфигурация:** Spring Boot автоматично конфигурира EntityManagerFactory, DataSource и TransactionManager, ако открие библиотеки като Hibernate и данни за връзка с базата.
- **Свойства за конфигурация в application.properties/yml:** Вместо persistence.xml конфигурацията се управлява чрез properties или yml файл.
- **Интеграция с Spring Data JPA:** Позволява лесно създаване на репозитории чрез интерфейси без нужда от boilerplate код.

```
1 spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
2 spring.datasource.username=dbuser
3 spring.datasource.password=dbpass
4 spring.datasource.driver-class-name=org.postgresql.Driver
5
6 # JPA/Hibernate
7 spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
8 spring.jpa.hibernate.ddl-auto=update # create, create-drop, validate, update
9 spring.jpa.show-sql=true
10 spring.jpa.properties.hibernate.format_sql=true
11
12 # Опции за performance
13 spring.jpa.properties.hibernate.jdbc.batch_size=50
14 spring.jpa.properties.hibernate.order_inserts=true
15 spring.jpa.properties.hibernate.order_updates=true
16
17 # Вторично кеширане (ако се използва)
18 spring.jpa.properties.hibernate.cache.use_second_level_cache=true
19 spring.jpa.properties.hibernate.cache.region.factory_class=org.hibernate.cache.jcache.JCacheRegionFactory
```

JPA конфигурации в Spring Boot

СВОЙСТВО	ОПИСАНИЕ
<code>spring.datasource.*</code>	Данни за базата данни (url, потребител, парола, драйвер).
<code>spring.jpa.database-platform</code>	SQL диалект, който Hibernate използва.
<code>spring.jpa.hibernate.ddl-auto</code>	Поведение за генериране или валидация на схемата (<code>create</code> , <code>update</code> , <code>validate</code> , <code>none</code>).
<code>spring.jpa.show-sql</code>	Показване на изпълнените SQL заявки в логовете.
<code>spring.jpa.properties.hibernate.*</code>	Допълнителни свойства за Hibernate.

Конфигуриране
на няколко
persistence unit
в Spring Boot

application.properties

properties

```
1 # Конфигурация за първа база (primaryDB)
2 spring.primary-datasource.jdbc-url=jdbc:postgresql://localhost:5432/primarydb
3 spring.primary-datasource.username=primary_user
4 spring.primary-datasource.password=primary_pass
5 spring.primary-datasource.driver-class-name=org.postgresql.Driver
6
7 # Конфигурация за втора база (secondaryDB)
8 spring.secondary-datasource.jdbc-url=jdbc:mysql://localhost:3306/secondarydb
9 spring.secondary-datasource.username=secondary_user
10 spring.secondary-datasource.password=secondary_pass
11 spring.secondary-datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

```

1 @Configuration
2 @EnableTransactionManagement
3 @EnableJpaRepositories(
4     basePackages = "com.example.primary.repository",
5     entityManagerFactoryRef = "primaryEntityManagerFactory",
6     transactionManagerRef = "primaryTransactionManager"
7 )
8 public class PrimaryDataSourceConfig {
9
10     @Bean
11     @ConfigurationProperties(prefix = "spring.primary-datasource")
12     public DataSource primaryDataSource() {
13         return DataSourceBuilder.create().build();
14     }
15
16     @Bean(name = "primaryEntityManagerFactory")
17     public LocalContainerEntityManagerFactoryBean primaryEntityManagerFactory(EntityManagerFactoryBuilder builder) {
18         return builder
19             .dataSource(primaryDataSource())
20             .packages("com.example.primary.model") // package с entity класовете
21             .persistenceUnit("primaryPU")
22             .build();
23     }
24
25     @Bean(name = "primaryTransactionManager")
26     public PlatformTransactionManager primaryTransactionManager(
27         @Qualifier("primaryEntityManagerFactory") EntityManagerFactory emFactory) {
28         return new JpaTransactionManager(emFactory);
29     }
30 }

```

Конфигуриране
на няколко
persistence unit
в Spring Boot

PrimaryDataSourceConfig

```

1 @Configuration
2 @EnableTransactionManagement
3 @EnableJpaRepositories({
4     basePackages = "com.example.secondary.repository",
5     entityManagerFactoryRef = "secondaryEntityManagerFactory",
6     transactionManagerRef = "secondaryTransactionManager"
7 })
8 public class SecondaryDataSourceConfig {
9
10     @Bean
11     @ConfigurationProperties(prefix = "spring.secondary-datasource")
12     public DataSource secondaryDataSource() {
13         return DataSourceBuilder.create().build();
14     }
15
16     @Bean(name = "secondaryEntityManagerFactory")
17     public LocalContainerEntityManagerFactoryBean secondaryEntityManagerFactory(EntityManagerFactoryBuilder builder) {
18         return builder
19             .dataSource(secondaryDataSource())
20             .packages("com.example.secondary.model") // package с entity класове
21             .persistenceUnit("secondaryPU")
22             .build();
23     }
24
25     @Bean(name = "secondaryTransactionManager")
26     public PlatformTransactionManager secondaryTransactionManager(
27         @Qualifier("secondaryEntityManagerFactory") EntityManagerFactory emFactory) {
28         return new JpaTransactionManager(emFactory);
29     }
30 }

```

Конфигуриране
на няколко
persistence unit
в Spring Boot

SecondaryDataSourceCo
nfig

Транзакции в JPA

Транзакцията в софтуерните системи (и по-конкретно в бази данни и JPA) е логическа единица работа, която трябва да бъде изпълнена изцяло или изобщо да не бъде изпълнена. Тя гарантира консистентност и надеждност на данните.

- **Основни свойства на транзакциите — ACID**

1. **Atomicity (Атомарност):** Всички операции в рамките на транзакцията се изпълняват изцяло или изобщо не.
2. **Consistency (Консистентност):** След успешна транзакция базата данни остава в консистентно състояние.
3. **Isolation (Изоляция):** Транзакциите се изпълняват така, че да не си пречат (почти като да са последователни).
4. **Durability (Издръжливост):** След като транзакцията е потвърдена (commit), промените остават трайни дори при сринове.

Транзакции в JPA

EntityManager предоставя методи за управление на транзакции:

- `entityManager.getTransaction().begin()`
- `entityManager.getTransaction().commit()`
- `entityManager.getTransaction().rollback()`

Този подход обикновено се използва при RESOURCE_LOCAL транзакции (управлявани от приложението).

Spring Framework осигурява декларативно управление на транзакциите чрез анотацията `@Transactional` и автоматично създава и управлява транзакционния контекст за методите, маркирани с тази анотация.

```
1 @Service
2 public class UserService {
3
4     @Autowired
5     private UserRepository userRepository;
6
7     @Transactional
8     public void createUser(User user) {
9         userRepository.save(user);
10        // други операции в рамките на транзакцията
11    }
12 }
```

JPQL-Java Persistence Query Language

JPQL (Java Persistence Query Language) е език за заявки, който се използва с JPA (Java Persistence API) в Java приложенията за достъп до релационни бази данни по обектно-ориентиран начин. JPQL ти позволява да пишеш заявки, които манипулират ентити обекти (тоест Java класове), а не директно таблици и колони от базата.

Основни характеристики на JPQL:

- **Обектно-ориентиран:** Работи с имената на ентити класовете и техните полета вместо с имената на таблиците и колоните.
- **Синтаксис, подобен на SQL:** Напомня синтаксиса на SQL, но с някои разлики, отнасящи се именно до работата с обекти.
- **Поддържа SELECT, UPDATE, DELETE:** Можеш да извличаш, изтриваш или обновяваш обекти чрез JPQL заявки.
- **JOIN, GROUP BY, ORDER BY:** Има възможности за обединяване (join) между ентити, подреждане, групиране и т.н.
- **Именовани и позиционни параметри:** Можеш да подаваш стойности на параметри по име (:name) или по позиция (?1).

Примери за JPQL

1. SELECT заявка (Извлича всички потребители над 18 години):

```
SELECT u FROM User u WHERE u.age > 18
```

2. JOIN (Намира всички поръчки (Order) за клиент от даден град):

```
SELECT o FROM Order o JOIN o.customer c WHERE c.city = :city
```

3. DTO проекция (Връща обекти UserDTO с определени данни):

```
SELECT new com.example.dto.UserDTO(u.name, u.email) FROM User u
```

4. BULK UPDATE (Променя цената на продуктите от дадена категория.):

```
UPDATE Product p SET p.price = p.price * 1.2 WHERE p.category = :cat
```

Изпълнение на JPQL заявка с EntityManager:

```
TypedQuery<User> query =  
    em.createQuery("SELECT u FROM User u WHERE u.name = :name", User.class);  
query.setParameter("name", "Ivan");  
List<User> users = query.getResultList();
```

JPQL

Предимства на JPQL:

- Пиши заявки така, както мислиш за обектите, а не за схемата на БД.
- Улеснява работата с релационни данни в обектно-ориентирани приложения.
- Позволява лесно използване на връзките между ентитита (OneToMany, ManyToOne).

Ограничения:

- Не всички възможности на SQL са налични (няма UNION и др.).
- BULK операциите (UPDATE/DELETE) заобикалят persistent контекста — трябва да си внимателен!

JPQL и Spring Boot чрез Spring Data JPA

В Spring Boot имаш няколко начина да използваш JPQL:

- **@Query анотация:** Можеш да дефинираш JPQL заявка директно в интерфейса на репозитори (Repository) чрез анотацията @Query.

```
1 @Query("SELECT u FROM User u WHERE u.email = :email")
2 User findByEmail(@Param("email") String email);
```

- **Custom Repository Methods:** Можеш да използваш именовани методи като `findByFirstNameAndLastName` и Spring Data JPA ще генерира нужната JPQL заявка автоматично.
- **EntityManager:** За по-сложни или динамични заявки можеш директно да използваш `EntityManager` и да изпълняваш JPQL чрез методите му.

Какви са предимствата?

- Заявките стават по-лесни за поддръжка, поради това, че работиш с имена на класове и техните полета.
- Можеш да използваш разнообразни техники за оптимизация като `FETCH JOIN`, `DTO` проекции, параметризирани заявки и др.
- Интеграцията със Spring Boot и Spring Data улеснява тестването, разширението и работата с промени по моделите.

Criteria API

Criteria API е част от Java Persistence API (JPA) и предоставя програмен, обектно-ориентиран начин за генериране на динамични заявки към базата данни вместо да пишеш заявки като стрингове (като при JPQL или SQL). В Spring Boot и Java EE приложения се използва, когато имаш нужда да генерираш заявки динамично – например, когато условията в заявката се определят по време на изпълнение въз основа на входни данни от потребителя.

- **Основни предимства на Criteria API**

- **Безопасност на типовете** – получаваш compile-time проверки, така че е по-трудно да направиш правописни грешки в имената на полета.
- **Динамичност** – можеш да комбинираш условия, JOIN-ове, сортиране и групиране по сложни правила, които са трудни или неудобни за реализиране с JPQL или named queries.
- **Лесна интеграция със Spring Data JPA и Hibernate.**
- **Платформено независимо** – работи с различни доставчици на JPA.

Criteria API в Spring Boot / JPA

```
@Autowired
private EntityManager em;

public List<User> findAdultUsersNamedIvan() {
    CriteriaBuilder cb = em.getCriteriaBuilder();

    CriteriaQuery<User> cq = cb.createQuery(User.class);
    Root<User> root = cq.from(User.class);

    Predicate agePredicate = cb.greaterThan(root.get("age"), 18);
    Predicate namePredicate = cb.equal(root.get("name"), "Иван");

    cq.select(root).where(cb.and(agePredicate, namePredicate));

    TypedQuery<User> query = em.createQuery(cq);
    return query.getResultList();
}
```

Основни стъпки

1. **Вземаш CriteriaBuilder** от EntityManager.
2. **Създаваш CriteriaQuery** за даден ентити клас.
3. **Дефинираш корена (Root)** на заявката – с кое ентити работиш.
4. **Добавяш условия (Predicate)** – where, join, orderBy и др.
5. **Създаваш заявка** от CriteriaQuery, изпълняваш я и взимаш резултата.

Criteria API

Преимущества спрямо JPQL:

- По-лесно можеш да комбинираш условия според подадени параметри (например ако търсиш по име ИЛИ по възраст, ИЛИ и по двете).
- Не пишеш заявки като текст, грешките се хващат по време на компилация.

Минуси:

- Кодът е по-описателен и става по-дълъг спрямо един @Query с JPQL или SQL.
- По-трудно се чете за прости заявки.

Кога да използваш Criteria API?

- Когато условията на заявка не са ясни по време на писане на кода (например при филтри по различни параметри от потребителска форма).
- Когато напредналата динамика и комбинацията на условията е задължителна.

Spring Data JPA

Spring Data JPA е модул към Spring Framework, който значително улеснява работата с релационни бази данни чрез Java Persistence API (JPA). Той автоматизира много от задачите, свързани с достъпа до данни – като създаване, четене, обновяване и изтриване (CRUD операции) – и позволява да се пишат по-малко boilerplate код.

Основни характеристики на Spring Data JPA:

- Използва интерфейси за репозитори и ги имплементира автоматично.
- Позволява дефиниране на заявки чрез именувани методи (findBy..., countBy... и др.).
- Интегрира се с различни JPA имплементации (като Hibernate).
- Поддържа писане на сложни заявки с @Query и JPQL/SQL.
- Поддържа пагинация, сортиране и динамични заявки.

Spring Data JPA

Основни интерфейси

- CrudRepository<T, ID> — базови CRUD операции.
- PagingAndSortingRepository<T, ID> — добавя пагинация и сортиране.
- JpaRepository<T, ID> — пълен набор операции, JPA-специфични методи.

Репозитори:

```
public interface UserRepository extends JpaRepository<User, Long> {  
    List<User> findByLastName(String lastName);  
    @Query("SELECT u FROM User u WHERE u.email = :email")  
    Optional<User> findByEmail(@Param("email") String email);  
}
```

Пагинация:

```
Page<User> page = userRepository.findByActive(true, PageRequest.of(0, 20, Sort.by("createdAt").descending()));
```

DTO проекции:

```
// Конструкторна проекция в JPQL  
@Query("SELECT new com.app.dto.UserDto(u.id, u.name) FROM User u WHERE ...")  
List<UserDto> findSummaries();
```

Spring Data JPA – именуване на методи

Именуването на методи в Spring Data JPA е една от най-полезните и удобни функции, които ти позволяват да дефинираш заявки (търсене, филтриране, сортиране) директно чрез името на метода в репозитори интерфейса, без да пишеш нито JPQL, SQL или @Query.

Как работи?

Spring Data JPA анализира името на метода и автоматично генерира подходяща заявка въз основа на него. В името могат да се използват имена на полета от ентити класа, ключови думи като And, Or, Between, LessThan, Like и други.

Основен синтаксис

- **findBy<FieldName>** — търси по дадено поле.
- **findBy<Field1>And<Field2>** — филтрира по няколко полета с AND.
- **findBy<Field1>Or<Field2>** — филтрира с OR.
- **countBy<Field>, existsBy<Field>** — брой/проверка за съществуване.

Spring Data JPA – именуване на методи

Ключови думи и оператори

- And, Or
- Is, Equals (по подразбиране)
- Between, LessThan, GreaterThan, After, Before
- Like, StartingWith, EndingWith, Containing
- In, NotIn
- OrderBy<Field>Asc/Desc
- Top, First, Distinct (ограничаване на резултата)

```
// User nonera: name, age, active, email
List<User> findByName(String name);
List<User> findByNameAndAge(String name, int age);
List<User> findByActiveTrue(); // по булеви
List<User> findByAgeGreaterThan(int age); // > age
List<User> findByAgeBetween(int min, int max);
List<User> findByNameLike(String pattern); // LIKE %pattern%
Optional<User> findByEmail(String email);
long countByActive(boolean active);
boolean existsByEmail(String email);
```

Именуване при вложени обекти (relations)

Ако User има връзка с Address:

```
List<User> findByAddress_City(String city);
List<User> findByAddress_CountryAndActiveTrue(String country);
```

Сортиране и пагинация

Може да добавиш Pageable или Sort като параметър:

```
Page<User> findByActive(boolean active, Pageable pageable);
List<User> findByNameOrderByAgeDesc(String name);
```

Specification

Specification е част от Spring Data JPA и представлява дизайн патърн (спецификация), който описва условия (филтри) за търсене/извличане на данни, използвайки типобезопасен и композируем начин чрез Criteria API. С него можеш динамично и удобно да изграждаш заявки с различни критерии, които се определят по време на изпълнение според нуждите на твоя слой за достъп до данни (например според избрани филтри в уеб форма или REST заявка).

Ключови предимства на Specification

- **Динамичност** – условията се определят в кода, често според входните данни.
- **Композиция** – можеш лесно да комбинираш няколко спецификации чрез `and()`, `or()`, `not()`.
- **Типова безопасност** – работи с Criteria API, което намалява риска от грешки при имената на полета.
- **Използваем с Paging и Sorting** – Specification работи с вградената пагинация и сортиране на Spring Data JPA.

Specification- пример

1. User Entity (пример)

```
java
1 @Entity
2 public class User {
3     @Id @GeneratedValue
4     private Long id;
5     private String name;
6     private Boolean active;
7     private Integer age;
8     // getters/setters
9 }
```

2. Филтър DTO

```
java
1 public class UserFilter {
2     private String name;
3     private Boolean active;
4     private Integer minAge;
5     // getters/setters
6 }
```

5. Използване в service или controller

```
@Autowired
private UserRepository userRepository;

public List<User> searchUsers(UserFilter filter) {
    Specification<User> spec = UserSpecification.filter(filter);
    return userRepository.findAll(spec);
}
```

3. Интерфейс на репозитория

```
1 public interface UserRepository extends JpaRepository<User, Long>, JpaSpecificationExecutor<User> {
2 }
```

4. Specification билдър (utility/helper или в service)

```
public class UserSpecification {

    public static Specification<User> filter(UserFilter filter) {
        return (root, query, cb) -> {
            List<Predicate> predicates = new ArrayList<>();

            if (filter.getName() != null && !filter.getName().isBlank()) {
                predicates.add(
                    cb.like(cb.lower(root.get("name")), "%" + filter.getName().toLowerCase() + "%")
                );
            }

            if (filter.getActive() != null) {
                predicates.add(
                    cb.equal(root.get("active"), filter.getActive())
                );
            }

            if (filter.getMinAge() != null) {
                predicates.add(
                    cb.greaterThanOrEqualTo(root.get("age"), filter.getMinAge())
                );
            }

            return cb.and(predicates.toArray(new Predicate[0]));
        };
    }
}
```

Как работи?

Създаваш един обект `UserFilter`, в който записваш само тези филтри, които са избрани/запълнени от потребител или фронтенд.

В спецификацията динамично се добавят условия към заявката само ако филтрите са попълнени.

Предаваш спецификацията на `findAll` и получения резултат автоматично включва само това, което отговаря на твоите условия.

Бележки

Можеш да комбинираш условия с `.and()` и `.or()` между няколко спецификации.

Можеш да добавяш пагинация, като използваш `findAll(Specification, Pageable)`.

Specification-ите са много мощни, когато филтрите не са предварително известни и се избира от потребителя динамично.

Благодаря за
вниманието